

Functional Requirements Document – Dragon Monster Game

Version: 1.0

Last Updated: July 23, 2024

Status: Active Development

Document Owner: Development Team

Table of Contents

1. [Executive Summary] (executive-summary)
2. [Project Scope] (project-scope)
3. [Functional Requirements by Module] (functional-requirements-by-module)
4. [User Stories] (user-stories)
5. [API Specifications] (api-specifications)
6. [Data Models] (data-models)
7. [Business Rules & Constraints] (business-rules--constraints)
8. [Non-Functional Requirements] (non-functional-requirements)
9. [Success Criteria] (success-criteria)

Executive Summary

Dragon Monster Game adalah aplikasi web berbasis browser yang menggabungkan:

- Backend: REST API dibangun dengan Spring Boot 4.0.5 (Java 25)
- Frontend: Single-page interface dengan vanilla JavaScript dan Canvas 2D rendering
- Database: PostgreSQL untuk persistent storage
- Gameplay: Turn-based RPG combat system dengan hero characters, enemies, dan dragon pets

Target Users:

- Casual game players
- Development portfolio demonstration
- Educational reference for Spring Boot + REST + Canvas gaming

Core Value Proposition:

Pemain dapat membuat hero, explore maps, bertarung dengan musuh, dan menyimpan progress mereka dalam save slots yang dapat dimuat kembali.

Project Scope

In Scope (Included)

Hero management (create, retrieve, list)
Dragon/Pet management
Turn-based combat system
Save slot functionality (create, read, update, delete)
Static HTML/JS frontend pages
Multi-map levels (Map 1, 2, 3)
Player character animation
Pet (dragon/companion) following mechanic
REST API endpoints for game operations

Out of Scope (Excluded)

Real-time multiplayer

User authentication/login system
Leaderboard/ranking system
In-app purchases or monetization
Mobile app native build
Persistent chat or social features
Advanced graphics (3D rendering)

Functional Requirements by Module

Hero Management Module

FR-1.1: Hero Retrieval

Requirement: System must provide endpoint to retrieve all heroes from database.

Acceptance Criteria:

- GET `/api/heroes` returns list of all heroes in JSON format
- Each hero includes: id, name, level, health, mana, attack, defense
- Response status: 200 OK
- Response format is consistent (JSON Array)
- Empty list returns `[]` if no heroes exist

Example Response:

```
```json
[
 {
 "id": 1,
 "name": "Wanderer Magician",
 "level": 1,
 "health": 100,
 "mana": 50,
 "attack": 15,
 "defense": 10,
 "experiencePoints": 0,
 "element": "FIRE"
 },
 {
 "id": 2,
 "name": "Knight Brave",
 "level": 1,
 "health": 150,
 "mana": 30,
 "attack": 20,
 "defense": 15,
 "experiencePoints": 0,
 "element": "WATER"
 }
]
```
```

FR-1.2: Get Hero by ID

Requirement: System must retrieve a specific hero by ID.

Acceptance Criteria:

- GET `/api/heroes/{id}` returns single hero object
- Returns 200 OK if hero exists
- Returns 404 Not Found if hero doesn't exist
- Response includes all hero properties

FR-1.3: Create Hero

Requirement: System must allow creation of new hero with initial stats.

Acceptance Criteria:

- POST `/api/heroes` accepts hero creation request
- Request body contains: name, element, health, mana, attack, defense
- System auto-generates: id, level=1, experiencePoints=0
- Returns 201 Created with created hero object
- Returns 400 Bad Request if required fields missing

FR-1.4: Hero Attributes

Requirement: Each hero has specific game attributes.

Hero Stats Definition:

| Attribute | Type | Initial Value | Max Value | Description |
|------------------|---------|-----------------------|-----------|---------------------------|
| id | Integer | Auto-generated | - | Unique identifier |
| name | String | User-provided | 50 chars | Hero display name |
| level | Integer | 1 | 100 | Current experience level |
| health | Integer | 100 | 999 | Current HP |
| maxHealth | Integer | 100 | 999 | Maximum HP |
| mana | Integer | 50 | 500 | Magic points |
| maxMana | Integer | 50 | 500 | Max magic points |
| attack | Integer | 15 | 999 | Physical damage output |
| defense | Integer | 10 | 999 | Physical damage reduction |
| experiencePoints | Integer | 0 | ∞ | Cumulative XP |
| element | Enum | FIRE/WATER/EARTH/WIND | - | Elemental type |

Dragon/Pet Management Module

FR-2.1: Dragon Retrieval

Requirement: System must provide endpoint to retrieve all available dragons/pets.

Acceptance Criteria:

- GET `/api/dragons` returns list of all dragons
- Each dragon includes: id, name, type, health, attack, speed, elementType
- Response status: 200 OK
- Dragons can be selected as player's pet companion

Example Response:

```
```json
[
 {
 "id": 1,
 "name": "Fire Dragon",
 "type": "dragon",
 "health": 80,
```

```

 "attack": 25,
 "speed": 10,
 "elementType": "FIRE",
 "rarity": "RARE"
 },
 {
 "id": 2,
 "name": "Unicorn",
 "type": "beast",
 "health": 60,
 "attack": 15,
 "speed": 15,
 "elementType": "LIGHT",
 "rarity": "RARE"
 }
]
...

```

#### FR-2.2: Dragon Selection

Requirement: Player can select a dragon as companion/pet.

##### Acceptance Criteria:

- Selected dragon appears alongside player in game
- Dragon follows player movement on map
- Dragon participates in combat if selected
- Only one dragon active at a time

#### FR-2.3: Dragon Attributes

Requirement: Each dragon has combat attributes.

##### Dragon Stats Definition:

Attribute	Type	Example	Description
id	Integer	1	Unique identifier
name	String	"Fire Dragon"	Display name
type	String	"dragon", "beast"	Pet category
health	Integer	80	Combat HP
attack	Integer	25	Attack power
speed	Integer	10	Turn order priority
elementType	Enum	FIRE, WATER, EARTH, WIND, LIGHT	Elemental type
rarity	Enum	COMMON, RARE, EPIC, LEGENDARY	Rarity tier

#### Combat System Module

##### FR-3.1: Combat Initiation

Requirement: Combat is triggered when player encounters enemy on map.

##### Acceptance Criteria:

- Combat starts automatically when player sprite collides with enemy
- Combat screen displays hero vs enemy stats
- Player can choose attack type (normal, special, defend)
- Enemy AI automatically selects actions

### FR-3.2: Hero Attack Endpoint

Requirement: Endpoint to process hero attack action in combat.

Endpoint: POST `/api/combat/hero-attack`

Request Body:

```
```json
{
  "heroId": 1,
  "enemyId": 5,
  "attackType": "normal"
}
```
```

Response Body:

```
```json
{
  "success": true,
  "damage": 20,
  "heroHealth": 85,
  "enemyHealth": 60,
  "experience": 0,
  "message": "Hero dealt 20 damage to enemy!",
  "isEnemyDefeated": false
}
```
```

Acceptance Criteria:

- POST request calculates damage based on hero attack stat + randomness
- Damage formula:  $(\text{heroAttack} + \text{random}(1-10)) - (\text{enemyDefense} / 2)$
- Enemy health decreases by damage amount
- Hero receives counter-damage if enemy survives
- Response includes updated health values
- System tracks combat state (ongoing, hero wins, enemy wins)

### FR-3.3: Damage Calculation

Requirement: Combat damage calculated using game balance formula.

Damage Formula:

```
```
baseDamage = attacker.attack + random(1, 10)
actualDamage = baseDamage - (defender.defense / 2)
elementMultiplier = checkElementAdvantage(attacker.element, defender.element)
finalDamage = actualDamage * elementMultiplier
```
```

Element Advantage Matrix:

| Attacker | Defender | Multiplier |
|----------|----------|------------|
| FIRE     | EARTH    | 1.5x       |
| WATER    | FIRE     | 1.5x       |
| EARTH    | WATER    | 1.5x       |
| WIND     | WATER    | 1.5x       |
| Default  | -        | 1.0x       |

#### FR-3.4: Combat End Condition

Requirement: Combat ends when hero or enemy health reaches 0.

##### Acceptance Criteria:

- Combat tracks both hero and enemy health
- When enemy health  $\leq 0$ : Hero wins, receives experience reward
- When hero health  $\leq 0$ : Hero loses, receives reduced experience
- Victory provides: Experience points, gold, loot
- Defeat allows player to retry or load save slot

##### Victory Rewards:

```
```json
{
  "experience": 50,
  "gold": 25,
  "items": ["health_potion", "mana_potion"],
  "levelUp": true
}
```
```

#### FR-3.5: Experience & Leveling

Requirement: Heroes gain experience from combat victories.

##### Acceptance Criteria:

- Victory grants experience points (50 XP base)
- Experience accumulates toward next level
- Level progression:  $\text{nextLevelXP} = \text{currentLevel} * 100$
- Stats increase on level up: health +10, attack +2, defense +1
- Notification displays when level up occurs

##### Leveling Example:

- Level 1  $\rightarrow$  Level 2: Requires 100 XP
- Level 2  $\rightarrow$  Level 3: Requires 200 XP
- Level 3  $\rightarrow$  Level 4: Requires 300 XP

#### Save Slot Management Module

##### FR-4.1: Save Slot Retrieval

Requirement: Player can view all saved game states.

Endpoint: GET `/api/slots`

##### Response:

```
```json
[
  {
    "id": 1,
    "playerName": "John Doe",
    "level": 5,
    "experience": 350,
    "heroId": 1,
    "dragonId": 1,
    "currentMap": "map.html",
  }
]
```

```
    "gold": 150,  
    "createdAt": "2024-07-23T10:30:00Z",  
    "updatedAt": "2024-07-23T12:45:00Z"  
  }  
]  
...
```

Acceptance Criteria:

- Returns all save slots for current player
- Includes save metadata (name, level, creation date)
- Returns 200 OK
- Returns empty array if no saves exist

FR-4.2: Create Save Slot

Requirement: Player can save current game state.

Endpoint: POST `/api/slots`

Request Body:

```
```json  
{
 "playerName": "John Doe",
 "level": 5,
 "experience": 350,
 "heroId": 1,
 "dragonId": 1,
 "currentMap": "map.html",
 "gold": 150
}
...
```

Acceptance Criteria:

- Creates new save slot in database
- Auto-generates: id, createdAt timestamp
- Returns 201 Created with full save slot object
- Validates required fields present
- Returns 400 Bad Request if validation fails

FR-4.3: Update Save Slot

Requirement: Player can overwrite existing save slot.

Endpoint: PUT `/api/slots/{id}`

Request Body:

```
```json  
{  
  "playerName": "John Doe",  
  "level": 7,  
  "experience": 650,  
  "heroId": 1,  
  "dragonId": 2,  
  "currentMap": "map2.html",  
  "gold": 300  
}  
...
```

Acceptance Criteria:

- Updates specific save slot by ID
- Updates `updatedAt` timestamp
- Returns 200 OK with updated object
- Returns 404 Not Found if slot doesn't exist
- Returns 400 Bad Request if invalid data

FR-4.4: Delete Save Slot

Requirement: Player can delete unwanted save slot.

Endpoint: DELETE `/api/slots/{id}`

Acceptance Criteria:

- Removes save slot from database
- Returns 200 OK on successful deletion
- Returns 404 Not Found if slot doesn't exist
- Cannot recover deleted saves

FR-4.5: Load Save Slot

Requirement: Player can load game state from save slot (frontend implementation).

Acceptance Criteria:

- Frontend fetches save slot data via GET `/api/slots/{id}`
- Restores player stats, hero, dragon, map position
- Continues gameplay from saved point
- No data loss on load

Enemy Management Module

FR-5.1: Enemy Data Model

Requirement: Enemies are entities that players encounter and battle.

Enemy Attributes:

```
```json
{
 "id": 5,
 "name": "Goblin Warrior",
 "health": 40,
 "maxHealth": 40,
 "attack": 10,
 "defense": 5,
 "level": 1,
 "experienceReward": 30,
 "goldReward": 15,
 "elementType": "EARTH",
 "rarity": "COMMON"
}
```
```


FR-5.2: Enemy Spawning

Requirement: Enemies appear on map for player to encounter.

Acceptance Criteria:

- Enemies spawn at random locations on map
- Multiple enemies can exist simultaneously
- Difficulty scales with player level (optional)
- Enemy stats vary by type and level

FR-5.3: Enemy AI

Requirement: Enemies take automated combat actions.

Acceptance Criteria:

- Enemies select actions: normal attack, special move, defend
- AI prioritizes damage when enemy health > 50%
- AI prioritizes defense when enemy health < 30%
- Enemy turn happens after hero turn

Frontend UI Module

FR-6.1: Game Main Page

Requirement: Main game interface accessible at `/game.html`

Acceptance Criteria:

- Loads game canvas (1280x720)
- Displays hero name and stats
- Shows current map/level
- Implements keyboard controls (arrow keys, W/A/D, Space)
- Renders player character sprite
- Renders pet/dragon companion

FR-6.2: Map Pages

Requirement: Multiple map levels accessible.

Maps Available:

- `/map.html` - Map 1 (Ruins)
- `/map2.html` - Map 2 (Forest)
- `/map3.html` - Map 3 (Castle)

Acceptance Criteria:

- Each map has unique parallax background layers
- Player can navigate freely
- Enemies spawn on each map
- Transitions between maps smooth
- Asset loading shows progress

FR-6.3: Setup/Configuration Page

Requirement: Player can configure game settings at `/setup.html`

Acceptance Criteria:

- Select hero from available list

- Select pet/dragon companion
- Configure difficulty level
- Create new game or load existing save
- Start gameplay button initiates game

FR-6.4: Player Controls

Requirement: Intuitive keyboard controls for gameplay.

Control Mappings:

| Key | Action | Context |
|-------------|--------------------|-------------|
| A or ← | Move Left | Gameplay |
| D or → | Move Right | Gameplay |
| W or ↑ | Jump | Gameplay |
| Space | Jump | Gameplay |
| Mouse Click | Select Menu Option | Menu |
| Enter | Confirm Selection | Menu/Dialog |

FR-6.5: Canvas Rendering

Requirement: Game renders using HTML5 Canvas 2D API.

Acceptance Criteria:

- 60 FPS target frame rate
- Smooth sprite animation (8 frames per animation cycle)
- Parallax scrolling backgrounds
- Collision detection for platforms
- Respawn at safe point on fall

Animation & Graphics Module

FR-7.1: Player Character Animation

Requirement: Player sprite shows different animations based on action.

Animation States:

| State | Sprite | Frames | Trigger |
|--------|-------------------|--------|-------------------|
| Idle | player_idle.png | 8 | Standing still |
| Run | player_run.png | 8 | Moving left/right |
| Jump | player_jump.png | 8 | In air |
| Attack | player_attack.png | 6 | During combat |

FR-7.2: Dragon/Pet Animation

Requirement: Dragon companion shows appropriate animations.

Animation States:

| State | Sprite | Frames | Trigger |
|----------|----------------------|--------|-------------------------|
| Idle Fly | fire_dragon_idle.png | 4 | Flying stationary |
| Fly | fire_dragon_fly.png | 4 | Following player in air |
| Walk | unicorn_run.png | 4 | Walking on ground |

FR-7.3: Parallax Scrolling

Requirement: Background layers move at different speeds creating depth.

Layer Speeds:

| Layer | Asset | Speed | Z-Index |
|------------|------------------|-------|---------|
| Sky | sky.png | 0x | -3 |
| Back Ruins | ruins_bg.png | 0.2x | -2 |
| Hills | hills&trees.png | 0.4x | -1 |
| Ruins | ruins2.png | 0.6x | 0 |
| Foreground | ruins.png | 0.8x | 1 |
| Ground | stones&grass.png | 2.0x | 2 |

User Stories

Epic 1: Hero Management

US-1.1: As a player, I want to view available heroes so I can choose one to play with.

- Acceptance: GET `/api/heroes` displays list with name, level, stats
- Priority: High
- Story Points: 3

US-1.2: As a player, I want to create a custom hero with a name I choose.

- Acceptance: POST `/api/heroes` creates hero with custom name
- Priority: High
- Story Points: 5

Epic 2: Combat System

US-2.1: As a player, I want to battle enemies and see combat results in real-time.

- Acceptance: POST `/api/combat/hero-attack` processes attack, returns damage dealt and health updated
- Priority: Critical
- Story Points: 8

US-2.2: As a player, I want to gain experience from defeating enemies and level up.

- Acceptance: Defeating enemy grants XP, hero levels when XP threshold reached, stats increase
- Priority: High
- Story Points: 5

US-2.3: As a player, I want to use strategy in combat (attack vs defend).

- Acceptance: Hero can choose attack type affecting damage output and defense
- Priority: Medium
- Story Points: 8

Epic 3: Pet/Dragon System

US-3.1: As a player, I want to select a dragon companion to follow me.

- Acceptance: Dragon appears on screen, follows player position, participates in combat
- Priority: Medium
- Story Points: 5

US-3.2: As a player, I want my dragon to have unique abilities in combat.

- Acceptance: Dragon can use special moves, different element types have advantages
- Priority: Low
- Story Points: 8

Epic 4: Save System

US-4.1: As a player, I want to save my progress so I can continue later.

- Acceptance: POST `/api/slots` creates save with current stats, can be retrieved
- Priority: High
- Story Points: 5

US-4.2: As a player, I want to load a previous save and continue from that point.

- Acceptance: GET `/api/slots/{id}` retrieves save, frontend restores game state
- Priority: High
- Story Points: 5

US-4.3: As a player, I want to manage multiple save slots.

- Acceptance: Can create, read, update, delete save slots independently
- Priority: Medium
- Story Points: 3

Epic 5: Gameplay Experience

US-5.1: As a player, I want to navigate through different maps and explore.

- Acceptance: Maps 1, 2, 3 accessible, smooth transitions between maps
- Priority: High
- Story Points: 8

US-5.2: As a player, I want smooth keyboard controls for movement.

- Acceptance: Arrow keys / WASD moves player, Space/W jumps, responsive
- Priority: High
- Story Points: 3

US-5.3: As a player, I want my character to animate smoothly while moving and fighting.

- Acceptance: Sprite animations play at 60 FPS, smooth transitions between states
- Priority: Medium
- Story Points: 5

API Specifications

Base URL

...

http://localhost:8081/api

...

Heroes Endpoints

GET /heroes Retrieve
all heroes.

Response: `200 OK`

```
```json
[
 {
 "id": 1,
 "name": "Wanderer Magician",
 "level": 1,
 "health": 100,
 "maxHealth": 100,
 "mana": 50,
 "maxMana": 50,
 "attack": 15,
 "defense": 10,
 "experiencePoints": 0,
 "element": "FIRE"
 }
]
```
```

POST /heroes Create
new hero.

Request:

```
```json
{
 "name": "New Hero",
 "element": "WATER",
 "health": 120,
 "mana": 40,
 "attack": 18,
 "defense": 12
}
```
```

Response: `201 Created`

Dragons Endpoints

GET /dragons
Retrieve all dragons.

Response: `200 OK`

```
```json
[
 {
 "id": 1,
 "name": "Fire Dragon",
 "type": "dragon",
 "health": 80,
 }
]
```

```
 "attack": 25,
 "speed": 10,
 "elementType": "FIRE",
 "rarity": "RARE"
 }
]
...
```

## Combat Endpoints

POST /combat/hero-attack  
Execute hero attack in combat.

Request:

```
```json  
{  
  "heroId": 1,  
  "enemyId": 5,  
  "attackType": "normal"  
}  
...
```

Response: `200 OK`

```
```json  
{
 "success": true,
 "damage": 22,
 "heroHealth": 85,
 "enemyHealth": 58,
 "experience": 0,
 "isEnemyDefeated": false,
 "message": "Hero dealt 22 damage!"
}
...
```

## Save Slots Endpoints GET

/slots  
Get all save slots.

Response: `200 OK`

```
```json  
[  
  {  
    "id": 1,  
    "playerName": "John",  
    "level": 5,  
    "experience": 350,  
    "heroId": 1,  
    "createdAt": "2024-07-23T10:30:00Z"  
  }  
]  
...
```

POST /slots Create
new save slot.

Request:

```
```json
{
 "playerName": "John",
 "level": 5,
 "experience": 350,
 "heroId": 1,
 "dragonId": 1,
 "gold": 150
}
```
```

Response: `201 Created`

GET /slots/{id} Get
specific save slot.

Response: `200 OK` or `404 Not Found`

PUT /slots/{id}
Update save slot.

Request: (Same as POST)

Response: `200 OK`

DELETE /slots/{id} Delete
save slot.

Response: `204 No Content`

Data Models

Hero Entity

```
```
Hero {
 - id: Long (Primary Key, Auto-generated)
 - name: String (max 50 chars)
 - level: Integer (default 1, max 100)
 - health: Integer
 - maxHealth: Integer (default 100)
 - mana: Integer
 - maxMana: Integer (default 50)
 - attack: Integer (default 15)
 - defense: Integer (default 10)
 - experiencePoints: Integer (default 0)
 - element: ElementType (FIRE, WATER, EARTH, WIND)
}
```
```

```
- createdAt: Timestamp
- updatedAt: Timestamp
}
```

```
Dragon Entity
...
```

```
Dragon {
- id: Long (Primary Key)
- name: String
- type: String (dragon, beast, etc)
- health: Integer
- attack: Integer
- speed: Integer
- elementType: ElementType
- rarity: RarityType (COMMON, RARE, EPIC, LEGENDARY)
}
```

```
Enemy Entity
...
```

```
Enemy {
- id: Long (Primary Key)
- name: String
- health: Integer
- maxHealth: Integer
- attack: Integer
- defense: Integer
- level: Integer
- experienceReward: Integer
- goldReward: Integer
- elementType: ElementType
- rarity: RarityType
}
```

```
SaveSlot Entity
...
```

```
SaveSlot {
- id: Long (Primary Key)
- playerName: String
- level: Integer
- experience: Integer
- heroId: Long (Foreign Key → Hero)
- dragonId: Long (Foreign Key → Dragon)
- currentMap: String
- gold: Integer
- createdAt: Timestamp
- updatedAt: Timestamp
}
```

Business Rules & Constraints

BR-1: Experience & Leveling

- Every level requires `level \* 100` experience points
- Maximum level is 100

- Level up grants: +10 health, +2 attack, +1 defense
- Experience is cumulative and never resets

BR-2: Combat Balance

- Minimum damage is 1 (never less)
- Element advantage multiplies damage by 1.5x
- Defense reduces damage by 50% of defense value
- Health cannot go below 0 (combat ends at 0)

BR-3: Hero Creation

- Hero name must be 3-50 characters
- Cannot create hero with duplicate name
- Initial stats cannot be negative
- Element type is mandatory

BR-4: Save Slots

- Maximum 10 save slots per player (future feature)
- Save name must be unique per player
- Save data is immutable after 24 hours (future feature)
- Cannot delete save while game is running

BR-5: Pet System

- Only one active pet at a time
- Pet cannot attack if hero is dead
- Pet level matches hero level
- Pet cannot exceed hero's experience

Non-Functional Requirements

NFR-1: Performance

- Response Time: All API responses within 200ms (p95)
- Frame Rate: Game renders at 60 FPS consistently
- Loading: Assets load within 2 seconds on initial load
- Throughput: Support 100 concurrent users

NFR-2: Availability

- Uptime: 99.5% availability SLA
- Maintenance Window: 1 hour monthly (planned)
- Recovery: RTO $\leq$ 1 hour, RPO $\leq$ 15 minutes

NFR-3: Security

- All database credentials via environment variables (no hardcoding)
- CORS configured for allowed origins only (not `\*` in production)
- Input validation on all API endpoints
- SQL injection protection via parameterized queries
- HTTPS enforced in production

NFR-4: Scalability

- Database connection pooling (HikariCP)
- Stateless API design (no session affinity required)
- Horizontal scaling possible with load balancer
- Database supports $\geq$ 1000 concurrent connections

NFR-5: Compatibility

- Browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- Java: JDK 25 (latest LTS: 21)
- Database: PostgreSQL 12+

- OS: Linux, macOS, Windows 10+

NFR-6: Maintainability

- Code follows Spring Boot best practices
- Clear separation: Controller → Service → Repository
- DTOs for request/response handling
- Comprehensive error handling with meaningful messages
- Logging at DEBUG, INFO, WARN, ERROR levels

NFR-7: Usability

- UI responds to inputs within 100ms
- Keyboard controls feel responsive and natural
- Error messages are clear and actionable
- Save/load operations under 1 second

Success Criteria

Functional Success Criteria

- [] All 7 hero endpoints return correct responses
- [] All 6 dragon endpoints functional
- [] Combat system calculates damage correctly per formula
- [] Save slots CRUD operations work end-to-end
- [] Enemy AI makes reasonable combat decisions
- [] Frontend loads all game pages without errors

Technical Success Criteria

- [] 95% code coverage on critical paths
- [] All unit tests pass
- [] API response time < 200ms (p95)
- [] Game maintains 60 FPS on modern browsers
- [] Zero security vulnerabilities (verified by static analysis)
- [] Database connection pooling properly configured

User Experience Success Criteria

- [] Player can start game and reach first combat in < 2 min
- [] Combat feels responsive and fair
- [] Animations are smooth without stuttering
- [] Save/load works reliably
- [] User can switch between maps without issues
- [] Pet companion behavior is intuitive

Deployment Success Criteria

- [] Application starts without manual intervention
- [] Database migrations run automatically on startup
- [] Environment variables properly sourced
- [] Logs indicate healthy startup sequence
- [] Health check endpoint returns 200 OK

Sign-Off

Document Status: Approved for Development

Version History:

| Version | Date | Author | Changes |
|---------|-------|--------|---------|
| -- | ----- | | |

| 1.0 | 2024-07-23 | Dev Team | Initial FRD creation |

Next Steps:

1. Create technical specifications (architecture, database schema)
2. Begin Sprint 1 development (Hero & Dragon modules)
3. Establish testing framework and CI/CD pipeline
4. Set up monitoring and alerting for production

End of Document